# Machine Learning Assignment 2

**BITS Pilani WILP · M.Tech AI/ML · Submission PDF template**

**IMPORTANT:** This document contains incomplete placeholders for links and BITS Virtual Lab evidence. Do not submit without completing them, reviewing the text in your own words, and validating your actual results.

## 1. GitHub repository link (required first)

**ADD VERIFIED CLICKABLE REPOSITORY URL HERE:** https://github.com/YOUR_USERNAME/YOUR_REPOSITORY

Repository must contain source files, requirements.txt, README.md, test_data.csv and five saved .joblib models.

## 2. Live Streamlit app link (required second)

**ADD VERIFIED LIVE APP URL HERE:** https://YOUR_APP_NAME.streamlit.app

## 3. Screenshot of BITS Virtual Lab execution (required third)

**INSERT YOUR OWN AUTHENTIC SCREENSHOT HERE.** Your screenshot should show the assignment executing in the BITS Virtual Lab environment. This proof cannot be generated from outside your lab account.

BITS Virtual Lab screenshot placeholder

INSERT GENUINE LAB SCREENSHOT BEFORE SUBMISSION

## 4. README content — a. Problem statement

Train and compare Logistic Regression, Decision Tree, KNN, Gaussian Naive Bayes and Support Vector Machine on one public binary classification dataset. Show Accuracy, AUC, Precision, Recall, F1 and MCC on the same held-out test records. Use saved models in an interactive Streamlit app, without retraining on uploads.

## b. Dataset description

Wisconsin Diagnostic Breast Cancer (WDBC), UCI Machine Learning Repository, distributed via scikit-learn. Public dataset: UCI WDBC source. It contains 569 instances and 30 numeric features. Target label 0 is malignant; 1 is benign (positive class for these metrics). Data split: 455 training and 114 test rows, stratified, random_state=42. Median imputation and standard scaling are learned on the training partition only.

## c. GitHub repository link

https://github.com/YOUR_USERNAME/YOUR_REPOSITORY — replace with your real link.

## d. Models used — measured comparison

| Model | Accuracy | AUC | Precision | Recall | F1 | MCC |
|---|---|---|---|---|---|---|
| Logistic Reg. | 0.9825 | 0.9954 | 0.9861 | 0.9861 | 0.9861 | 0.9623 |
| Decision Tree | 0.9035 | 0.9373 | 0.9420 | 0.9028 | 0.9220 | 0.7969 |
| KNN | 0.9561 | 0.9788 | 0.9589 | 0.9722 | 0.9655 | 0.9054 |
| Naive Bayes | 0.9298 | 0.9868 | 0.9444 | 0.9444 | 0.9444 | 0.8492 |
| SVM | 0.9825 | 0.9950 | 0.9861 | 0.9861 | 0.9861 | 0.9623 |

Metrics use the same 114 held-out rows; AUC is calculated from positive-class probabilities. Precision, recall and F1 treat class 1 (benign) as positive.

## Model-wise observations

**Logistic Regression:** Excellent linear baseline with the highest AUC (tie-best accuracy/F1/MCC). Standardization helps optimization and coefficients remain interpretable.

**Decision Tree:** Lowest F1 and MCC here; its single-tree partitions are easy to explain but are less stable on this particular held-out split.

**KNN:** Strong F1 and recall on standardized feature space, though prediction cost grows with training set size.

**Naive Bayes:** Good AUC but lower classification accuracy; Gaussian conditional-independence assumption may be imperfect for correlated tumor measurements.

**Support Vector Machine:** Ties Logistic Regression for accuracy, F1, and MCC. RBF kernel can learn nonlinear boundaries but needs feature scaling.

**Overall winner:** Logistic Regression, tied with SVM on accuracy, F1 and MCC but with slightly higher AUC. This conclusion applies to this fixed split.

## e. Reproducibility and evaluation methodology

Dataset loaded via scikit-learn; stratified hold-out split; all five scikit-learn pipelines fitted on training rows only. Each pipeline stores median imputer, StandardScaler and fitted estimator. Use `python train_models.py` to reproduce and `streamlit run app.py` to launch. App loads saved .joblib models, validates CSV schema, predicts and renders per-model metrics, report, ROC and confusion matrix; uploaded test rows are never used for fitting.

## f. Repository structure and files

app.py; train_models.py; requirements.txt; README.md; test_data.csv; training_data.csv; evaluation_metrics.csv; .gitignore; model/metadata.json; model/logistic_regression.joblib; model/decision_tree.joblib; model/knn.joblib; model/naive_bayes.joblib; model/svm.joblib.

## g. Deployment steps

Push all files to your GitHub repository. Visit https://streamlit.io/cloud, sign in with GitHub, select New app, select repository/branch/main and entry point app.py, then Deploy. Verify link in a fresh browser and test data upload. Replace placeholders on this PDF with genuine URLs.

## h. Final checklist

Verify source and trained models are present; both public URLs work; Streamlit launches; CSV test upload works; model selector works; six metrics, confusion matrix and all-model comparison display; personal BITS Virtual Lab execution screenshot is inserted; README content is included in the PDF.

## i. Limitations and reflection

One fixed held-out split gives an estimate, not a guarantee of future performance. Performance depends on dataset, split and hyperparameters; test data must remain outside training. An independent improvement could add cross-validation on training rows, malignant-class recall analysis and original commentary. This system is not for clinical diagnosis.

**NOTE:** The PDF summarizes the full README sections. For strict compliance, ensure the final submitted PDF includes the entire final customized README content, not only this condensed version.

## Classification Model Benchmark — ML Assignment 2 (Reference Project)

> **Academic-integrity notice:** This project is a learning scaffold, not a ready-to-submit personal assignment. Run it in BITS Virtual Lab, inspect the source and model decisions, make your own meaningful enhancements and write your own conclusions before submitting. Do not claim the provided work or screenshots as work you performed when you did not.

## a. Problem statement

Build and compare five supervised classifiers on **one public classification dataset**, assess the same held-out samples using accuracy, ROC AUC, precision, recall, F1, and Matthews correlation coefficient (MCC), then publish an interactive Streamlit evaluator that uses **previously saved fitted models**, never fitting on uploaded test data.

## b. Dataset description

- **Dataset:** Wisconsin Diagnostic Breast Cancer (WDBC), distributed by scikit-learn, originally sourced from the UCI Machine Learning Repository.

- **UCI original:** https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic

- **Instances:** 569 (required: at least 500); **numeric input features:** 30 (required: at least 12).

- **Task:** Binary classification. `target=0` means malignant, `target=1` means benign (positive class in the evaluation metrics).

- **Data basis:** 30 real-valued cell-nuclei measurements derived from digitized images of breast mass fine-needle aspirates. No extra features are engineered.

- **Split:** stratified 80% training (455 samples) and 20% testing (114 samples), `random_state=42`.

- **Preprocessing:** median imputation, StandardScaler, then classifier; all fitted **only on training rows** via scikit-learn pipelines. No test leakage.

- **Data files:** `training_data.csv` and `test_data.csv` are generated from the published dataset. Only test data should be uploaded to the app.

- **Caution:** Benchmark exercise only; no clinical use or medical diagnosis is intended.

## c. GitHub Repository Link

**REPLACE WITH YOUR PUBLIC REPOSITORY URL:** `https://github.com/YOUR_USERNAME/YOUR_REPOSITORY`

**Streamlit Community Cloud app:** `https://YOUR_APP_NAME.streamlit.app` (replace after deployment).

## d. Models used and comparison table

| ML Model Name | Accuracy | AUC | Precision | Recall | F1 | MCC |
|---|---|---|---|---|---|---|
| Logistic Regression | 0.9825 | 0.9954 | 0.9861 | 0.9861 | 0.9861 | 0.9623 |
| Decision Tree | 0.9035 | 0.9373 | 0.9420 | 0.9028 | 0.9220 | 0.7969 |
| KNN | 0.9561 | 0.9788 | 0.9589 | 0.9722 | 0.9655 | 0.9054 |
| Naive Bayes | 0.9298 | 0.9868 | 0.9444 | 0.9444 | 0.9444 | 0.8492 |
| Support Vector Machine | 0.9825 | 0.9950 | 0.9861 | 0.9861 | 0.9861 | 0.9623 |

All metrics are computed on the same untouched held-out **114-row test set**, with class **1 (benign)** as positive. The ROC AUC uses predicted class probabilities rather than predicted class labels. Test-set results are not a substitute for external validation.

## Model-wise observations

| ML Model Name | Observation |
|---|---|
| Logistic Regression | Excellent linear baseline with the highest AUC (tie-best accuracy/F1/MCC). Standardization helps optimization and coefficients remain interpretable. |
| Decision Tree | Lowest F1 and MCC here; its single-tree partitions are easy to explain but are less stable on this particular held-out split. |

| KNN | Strong F1 and recall on standardized feature space, though prediction cost grows with training set size. |
| Naive Bayes | Good AUC but lower classification accuracy; Gaussian conditional-independence assumption may be imperfect for correlated tumor measurements. |
| Support Vector Machine | Ties Logistic Regression for accuracy, F1, and MCC. RBF kernel can learn nonlinear boundaries but needs feature scaling. |

**Overall winner (tie-break: highest F1, then AUC): Logistic Regression.** Logistic Regression and SVM achieve equal test F1, accuracy and MCC; Logistic Regression has slightly higher AUC on this split. These results depend on the fixed data split and hyperparameters. For real malignant-case sensitivity, also examine the classification report and confusion matrix: benign is the positive label here by dataset convention.

## e. Reproducible training and evaluation

1. Create a Python 3.11+ environment (preferably in the BITS Virtual Lab): `python -m venv .venv`.
2. Activate it: Linux/macOS `source .venv/bin/activate`; Windows PowerShell `.venv\Scripts\Activate.ps1`.
3. Install dependencies: `pip install -r requirements.txt`.
4. To independently reproduce saved models and CSV: `python train_models.py`.
5. Start the evaluator: `streamlit run app.py`.
6. Select a model from the left sidebar, inspect results on bundled test data, then upload **`test_data.csv`** using the CSV option; enable all-model comparison.
7. Capture **a genuine screenshot with BITS Virtual Lab visible** after executing the project there.

**Important:** `train_models.py` is the only code that calls `.fit()`. The Streamlit app loads `.joblib` files and calls `.predict()` and `.predict_proba()` on the supplied test CSV. Model pipelines preserve training-fitted imputers and scalers. Only load model files from your own trusted repository; joblib uses Python pickle serialization.

## f. Repository structure

```text
ML_Assignment_2_Reference_Project/
■■■ app.py                      # Interactive dashboard (no fitting)
■■■ train_models.py             # Reproducible dataset split and training
■■■ requirements.txt
■■■ README.md
■■■ test_data.csv               # 114 held-out labeled test rows
■■■ training_data.csv           # 455 training rows, reproducibility
■■■ evaluation_metrics.csv      # Real measured model comparison
■■■ .gitignore
■■■ model/
    ■■■ metadata.json          # Features, target, model filenames, split
    ■■■ logistic_regression.joblib
    ■■■ decision_tree.joblib
    ■■■ knn.joblib
    ■■■ naive_bayes.joblib
    ■■■ svm.joblib
```

## g. Deploy on Streamlit Community Cloud

1. Create your own GitHub repository; commit the project, including all five `model/*.joblib` artifacts, `test_data.csv`, and `requirements.txt`.
2. At https://streamlit.io/cloud, sign in with GitHub and select **New app**.
3. Choose the public repository, the `main` branch and entry point `app.py`, then deploy.
4. Open the live link in a fresh browser, test the bundled CSV and an upload of `test_data.csv`, and validate that five results and confusion matrices display.
5. Update this README and the submission PDF with the real live URL and repository URL.

## h. Required PDF submission and validation

Submit **one PDF** in this required order: (1) clickable GitHub repository link, (2) clickable live Streamlit app link, (3) **your own screenshot of execution in BITS Virtual Lab**, then (4) the full README content with dataset description, model metrics, model-wise observations and overall winner. A separate `submission_template.pdf` is included for the layout, but **you must replace its placeholders and insert your genuine lab screenshot**.

**Pre-submit checks:** verify both live links publicly open, all five trained artifact files are committed, test upload includes 30 original feature names plus `target`, no retraining occurs in the app, screenshot proves BITS Virtual Lab execution, and README/PDF match your actual final run. Submission deadline: **8 November 2026, 23:59 (per assignment sheet)**. The brief states **15 marks total (10 implementation/repository + 4 Streamlit + 1 lab screenshot)**.

## i. Limitations and possible independent improvements

This is a single train/test split, not nested cross-validation. Metrics are on a small test set; performance may fluctuate with the split. No hyperparameter tuning was performed on held-out test data. For an original extension, add training-only cross-validation, per-class comparison emphasizing malignant recall, a model-card page, or a custom feature significance visualization and explain your decisions in your own words.